

1. Business Overview

In [1]: `import duckdb`

```
con = duckdb.connect("olist.duckdb")
```

In [2]: `con.sql("SHOW TABLES").show()`

name varchar
customers geolocation order_items orders payments products reviews sellers translation

In [3]: *# How many customer records exist?*

```
con.sql("""SELECT COUNT(customer_id) AS total_customers FROM customers;""").show()
```

total_customers int64
99441

In [4]: *# How many unique customers exist?*

```
con.sql("""SELECT COUNT(Distinct customer_unique_id) AS total_customers FROM custom
```

total_customers int64
96096

In [5]: *# Is there a difference between customer records and unique customers?*

```
con.sql("""select count(customer_id)-count(distinct customer_unique_id) as duplicat
```

duplicate_customer_count int64
3345

In [6]: *# How many orders exist?*
 con.sql("""select count(*) from orders;""")

Out[6]:

count_star() int64
99441

In [7]: *# How many successfully delivered orders exist?*
 con.sql("""select count(*) as suc_order_count from orders where order_status = 'del

suc_order_count int64
96470

In [8]: *# Total Products*
 con.sql("""select count(product_id) as tot_prd_cnt from products;""").show()

tot_prd_cnt int64
32951

In [9]: *# Total Sellers*
 con.sql("""select count(seller_id) as tot_seller_cnt from sellers;""").show()

tot_seller_cnt int64
3095

In [10]: *# Total Product Categories*
 con.sql("""select count(distinct product_category_name) as tot_prd_cat from product

tot_prd_cat int64
73

```
In [11]: # Cancelled Orders
con.sql("""select count(order_id) as cancel_order_count, round((cancel_order_count/
```

Out[11]:

cancel_order_count int64	cancelled_per double
625	0.63

```
In [12]: # unavailable Orders
con.sql("""select count(order_id) from orders where order_status = 'unavailable';""
```

Out[12]:

count(order_id) int64
609

```
In [13]: # Delivery Success Rate
con.sql("""select count(*) as suc_order_count, round((suc_order_count/(select count
```

suc_order_count int64	del_suc_rate double
96470	97.01

Exploratory Data Analysis (EDA)

Business Overview

Marketplace Scale

Customer Base

- Total Customer Records: **99,441**
- Unique Customers: **96,096**
- Difference: **3,345**

Orders

- Total Orders: **99,441**

Products

- Total Products: **32,951**
- Total Product Categories: **73**

Sellers

- Total Sellers: **3,095**
-

Order Health

Successfully Delivered Orders

- Successfully Delivered Orders: **96,470**
- Delivery Success Rate: **97.01%**

Cancelled Orders

- Cancelled Orders: **625**
- Cancellation Rate: **0.63%**

Unavailable Orders

- Unavailable Orders: **609**
 - Unavailable Order Rate: **0.61%**
-

Key Insights

Customer Analysis

- The platform contains **99,441 customer records** but only **96,096 unique customers**.
- A difference of **3,345 records** indicates that some customers have made purchases using multiple customer IDs.
- Customer-level analysis should therefore be performed using **customer_unique_id** rather than **customer_id**.

Marketplace Size

- The marketplace serves more than **96K unique customers**.

- Nearly **100K orders** have been placed on the platform.
- The catalog contains more than **32K products** across **73 categories**.
- Over **3K sellers** participate in the marketplace.
- These figures indicate a large and mature e-commerce ecosystem.

Order Fulfillment Performance

- The platform successfully delivered **97.01%** of all orders.
- This indicates a highly efficient order fulfillment process.

Order Failure Analysis

- Only **0.63%** of orders were cancelled.
- Only **0.61%** of orders became unavailable.
- Combined order failure rate is approximately **1.24%**, suggesting strong operational performance.

Product Catalog

- The marketplace offers a diverse catalog of **32,951 products** across **73 categories**.
- This diversity enables broad customer reach across multiple product segments.

Seller Ecosystem

- The marketplace consists of **3,095 sellers**.
 - Further analysis is required to determine whether sales are evenly distributed or concentrated among a small group of sellers.
-

Summary

The Olist marketplace demonstrates:

- A large customer base.
- Strong order volume.
- High order fulfillment efficiency.
- Low cancellation and unavailability rates.
- A diverse product catalog.
- A sizable seller network.

Overall, the platform appears operationally healthy and suitable for deeper customer, revenue, seller, and product analysis.

2. Time-Based Analysis

In [14]: *# How many orders were placed each year?*

```
year_orders = con.sql("""
select
    year(order_purchase_timestamp) as year,
    count(order_id) as order_count
from
    orders
group by
    year
order by
    year;
""").df()
year_orders
```

Out[14]:

	year	order_count
--	------	-------------

0	2016	329
---	------	-----

1	2017	45101
---	------	-------

2	2018	54011
---	------	-------

In [15]: *# How many orders were placed each month?*

```
month_orders = con.sql("""
select
    month(order_purchase_timestamp) as month,
    monthname(order_purchase_timestamp) as monthname,
    count(order_id) as order_count
from
    orders
group by
    month,
    monthname
order by
    month;
""").df()
month_orders
```

Out[15]:

	month	monthname	order_count
0	1	January	8069
1	2	February	8508
2	3	March	9893
3	4	April	9343
4	5	May	10573
5	6	June	9412
6	7	July	10318
7	8	August	10843
8	9	September	4305
9	10	October	4959
10	11	November	7544
11	12	December	5674

In [16]: *# How many orders were placed each month in each year?*

```
year_month_orders = con.sql("""
select
    year(order_purchase_timestamp) as year,
    month(order_purchase_timestamp) as month,
    monthname(order_purchase_timestamp) as monthname,
    count(order_id) as order_count
from
    orders
group by
    year,
    month,
    monthname
order by
    year,
    month;
""").df()
```

In [17]: *# Which month recorded the highest number of orders?*

```
con.sql("""select month(order_purchase_timestamp) as month, monthname(order_purchas
```

month	monthname	max_order_count
int64	varchar	int64
8	August	10843

In [18]: *# Which month recorded the lowest number of orders?*

```
con.sql("""select month(order_purchase_timestamp) as month, monthname(order_purchas
```

month int64	monthname varchar	min_order_count int64
9	September	4305

```
In [19]: year_month_orders["year_month"] = (
    year_month_orders["year"].astype(str)
    + "-"
    + year_month_orders["month"].astype(str).str.zfill(2)
)
year_month_orders.head()
```

```
Out[19]:
```

	year	month	monthname	order_count	year_month
0	2016	9	September	4	2016-09
1	2016	10	October	324	2016-10
2	2016	12	December	1	2016-12
3	2017	1	January	800	2017-01
4	2017	2	February	1780	2017-02

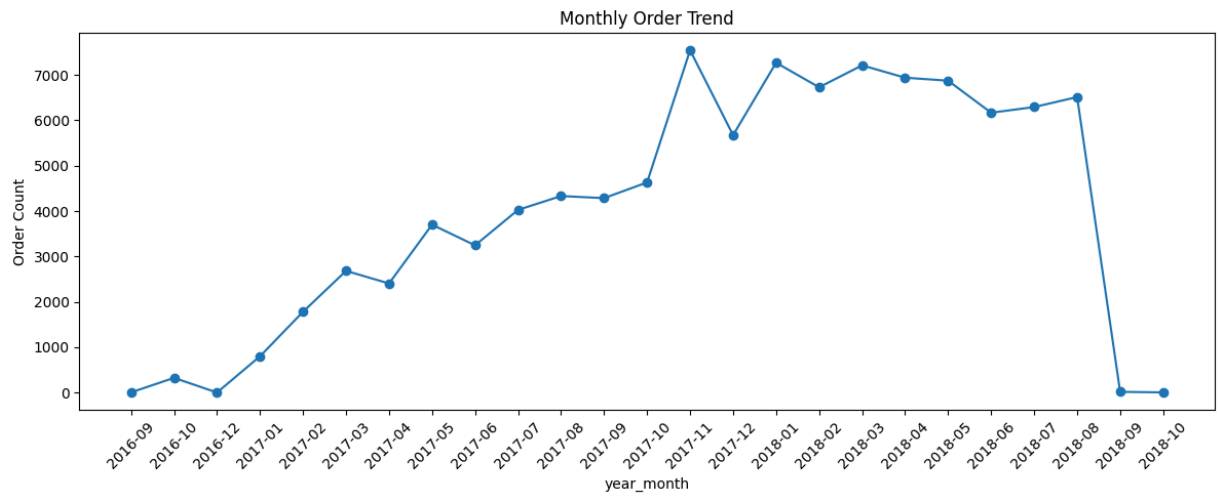
```
In [20]: # Is the order volume increasing over time?
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

plt.plot(
    year_month_orders["year_month"],
    year_month_orders["order_count"],
    marker="o"
)

plt.xticks(rotation=45)
plt.title("Monthly Order Trend")
plt.xlabel("year_month")
plt.ylabel("Order Count")

plt.tight_layout()
plt.show()
```



In [21]: *# Is the order volume increasing over time?*
`import matplotlib.pyplot as plt`

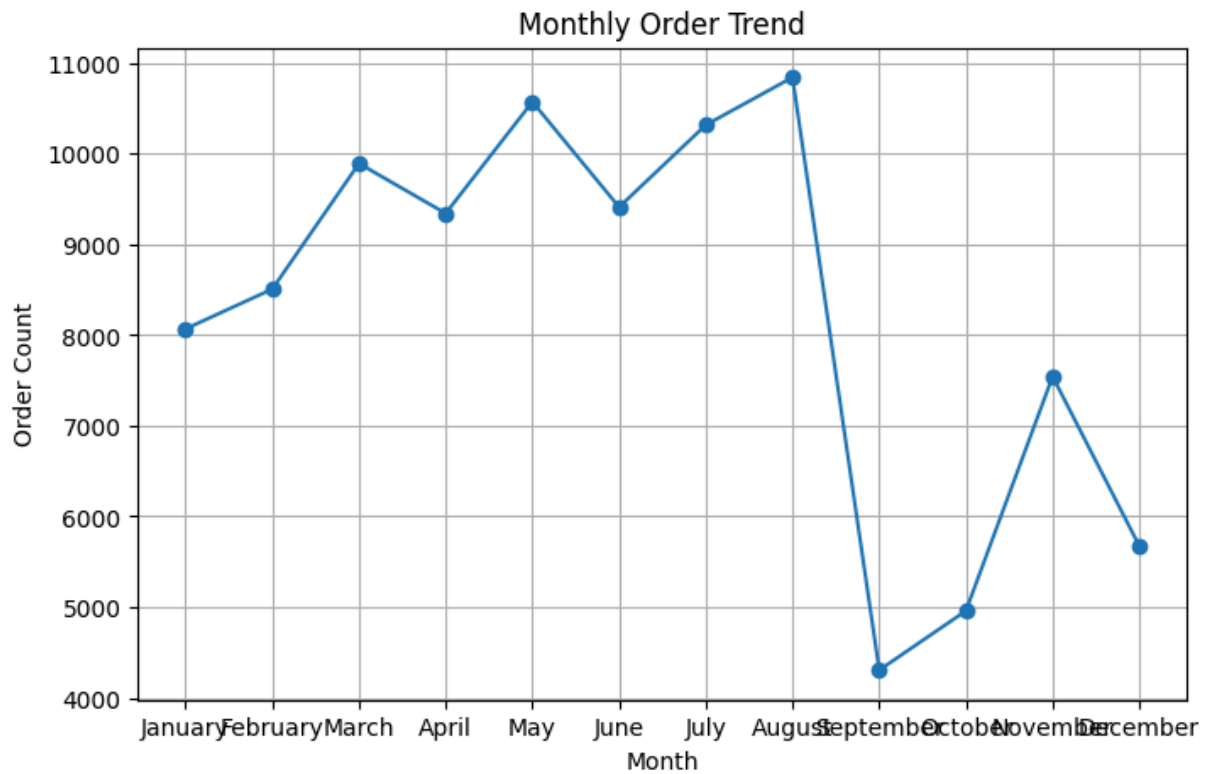
```
plt.figure(figsize=(8,5))

plt.plot(
    month_orders["monthname"],
    month_orders["order_count"],
    marker="o"
)

plt.title("Monthly Order Trend")
plt.xlabel("Month")
plt.ylabel("Order Count")

plt.grid(True)

plt.show()
```



In [22]: *# Is the order volume increasing over time?*

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(
    year_orders["year"],
    year_orders["order_count"],
    marker="o"
)
```

```
plt.title("Yearly Order Trend")
```

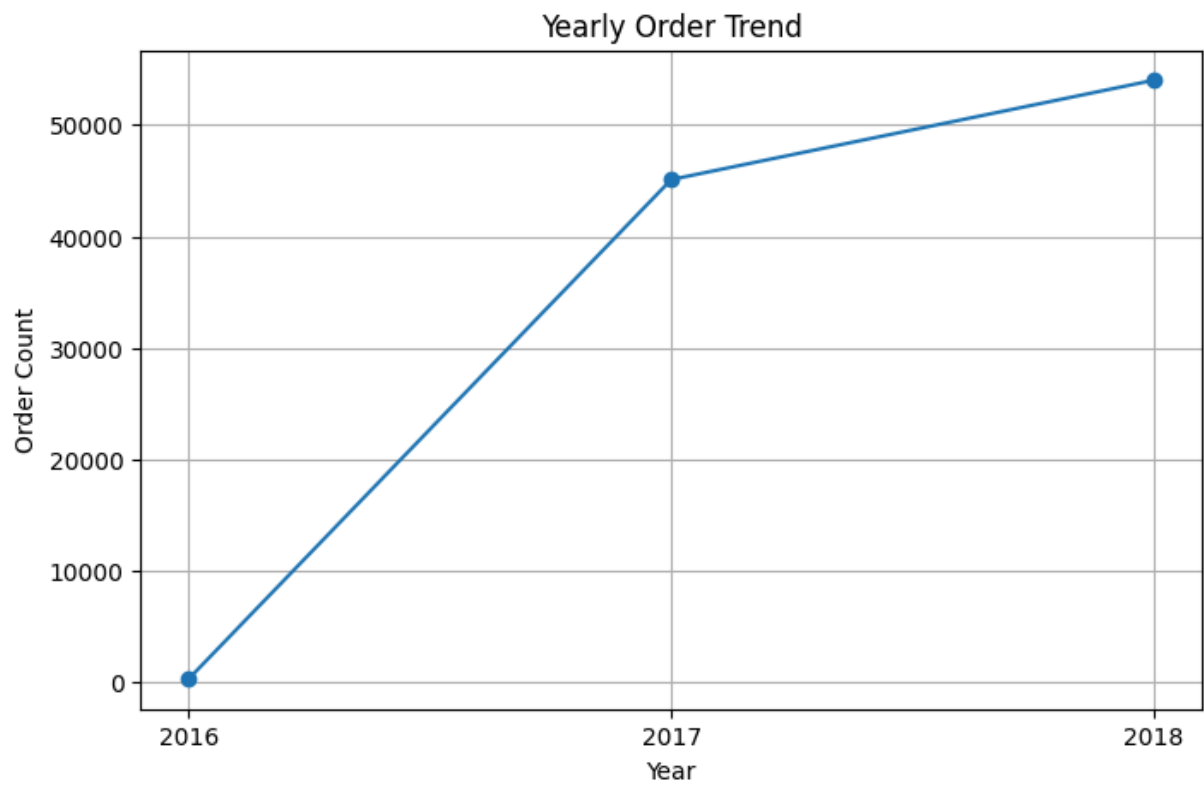
```
plt.xlabel("Year")
```

```
plt.ylabel("Order Count")
```

```
plt.xticks(year_orders["year"])
```

```
plt.grid(True)
```

```
plt.show()
```



In []:

In []:

In []:

In [23]: `con.sql("""select * from products;""").df()`

Out[23]:

	product_id	product_category_name	product_name
0	1e9e8ef04dbcff4541ed26657ea517e5		perfumaria
1	3aa071139cb16b67ca9e5dea641aaa2f		artes
2	96bd76ec8810374ed1b65e291975717f		esporte_lazer
3	cef67bcfe19066a932b7673e239eb23d		bebes
4	9dc1a7de274444849c219cff195d0b71		utilidades_domesticas
...	...		...
32946	a0b7d5a992ccda646f2d34e418fff5a0		moveis_decoracao
32947	bf4538d88321d0fd4412a93c974510e6	construcao_ferramentas_iluminacao	
32948	9a7c6041fa9592d9d9ef6cfe62a71f8c		cama_mesa_banho
32949	83808703fc0706a22e264b9d75f04a2e	informatica_acessorios	
32950	106392145fca363410d287a815be6de4		cama_mesa_banho

32951 rows × 9 columns



Time-Based Analysis - Key Insights

Business Growth

- Order volume increased significantly throughout 2017.
- Monthly orders grew from approximately 800 orders in January 2017 to over 7,500 orders in November 2017.
- This indicates strong marketplace expansion and increasing customer adoption.

Peak Demand Period

- The highest order volume was observed in November 2017.
- This may be associated with Black Friday and year-end promotional activity.

Business Maturity

- During 2018, monthly order volume stabilized between approximately 6,000 and 7,200 orders.
- This suggests that the marketplace transitioned from a rapid growth phase into a more stable operating phase.

Data Limitation

- September 2018 and October 2018 contain very low order counts.
- These months appear to contain partial data and were excluded from business trend interpretation.

Overall Conclusion

- The marketplace demonstrated strong growth during 2017 and sustained high transaction volumes throughout 2018.
- Order demand appears healthy and stable, indicating a mature and active e-commerce platform.

Revenue Analysis

```
In [24]: # What is the total revenue generated?
con.sql("""
select
    round(sum(oi.price),2) as total_revenue
from
    order_items oi
""").show()
```

total_revenue
double
13591643.7

```
In [25]: # How many orders contributed to revenue?
con.sql("""
select
    count(distinct order_id) as prod_orders
from
    order_items;
""").show()
```

prod_orders
int64
98666

```
In [26]: # How many customers contributed to revenue?
con.sql("""
select
    count(distinct c.customer_unique_id) as prod_customers
from
    customers c
join
    orders o on c.customer_id = o.customer_id
```

```
join
  order_items oi on o.order_id = oi.order_id;
""").show()
```

prod_customers
int64
95420

```
In [27]: # What is the Average Order Value (AOV)?
con.sql("""
select
  sum(price)/count(distinct order_id) as Average_Order_Revenue
from
  order_items;
""").show()
```

Average_Order_Revenue
double
137.75407637903828

```
In [28]: # What is the average revenue per customer?
con.sql("""
select
  round(sum(oi.price)/count(distinct c.customer_unique_id),2) as Avg_Rvn_per_cust
from
  customers c
join
  orders o on o.customer_id = c.customer_id
join
  order_items oi on o.order_id = oi.order_id;
""").show()
```

Avg_Rvn_per_cust
double
142.44

Revenue Loss Analysis

```
In [29]: # How much potential revenue was lost due to cancelled and unavailable orders?
con.sql("""
select
  round(sum(oi.price),2) as lost_revenue
from
```

```

    orders o
join
    order_items oi on o.order_id = oi.order_id
where
    o.order_status in ('canceled', 'unavailable');
""").show()

```

lost_revenue
double
97242.96

In [30]: *# What percentage of total potential revenue was lost?*

```

con.sql("""
select round((
    (select sum(oi.price)
    from orders o join order_items oi on o.order_id = oi.order_id
    where o.order_status IN ('canceled', 'unavailable')) * 100.0 / sum(price)),2)
    as lost_rev_per
from
    order_items;
""").show()

```

lost_rev_per
double
0.72

In [31]: *# Which states contribute most to lost revenue?*

```

con.sql("""
with lost_rvn as (
select
    c.customer_state,
    round(sum(oi.price),2) as lost_revenue
from
    customers c
join
    orders o on c.customer_id = o.customer_id
join
    order_items oi on o.order_id = oi.order_id
where
    o.order_status IN ('canceled', 'unavailable')
group by
    c.customer_state
order by
    lost_revenue desc)

select
    customer_state,
    lost_revenue
from

```

```
lost_rvn
Limit 1;
""").show()
```

customer_state varchar	lost_revenue double
SP	39087.83

```
In [32]: # Which product categories contribute most to lost revenue?
con.sql("""
select
    p.product_category_name,
    round(sum(oi.price),2) as lost_revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
join
    products p on p.product_id=oi.product_id
where
    o.order_status IN ('canceled', 'unavailable')
group by
    p.product_category_name
order by
    lost_revenue desc
limit 1;
""")
```

Out[32]:

product_category_name varchar	lost_revenue double
cool_stuff	14520.36

Revenue Trend Analysis

```
In [33]: # How much revenue was generated each year?
yearly_revenue = con.sql("""
select
    year(o.order_purchase_timestamp) as year,
    sum(oi.price) as total_revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    year;
""").df()
yearly_revenue
```

```
Out[33]:
```

	year	total_revenue
0	2016	4.978592e+04
1	2017	6.155807e+06
2	2018	7.386051e+06

```
In [34]: yearly_revenue.dtypes
```

```
Out[34]: year          int64
total_revenue    float64
dtype: object
```

```
In [35]: # How much revenue was generated each month?
mnth_rev = con.sql("""
select
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    round(sum(oi.price),2) as total_revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    month,
    month_name
order by
    month;
""").df()
mnth_rev
```

```
Out[35]:
```

	month	month_name	total_revenue
0	1	January	1070343.23
1	2	February	1091481.73
2	3	March	1357557.74
3	4	April	1356574.98
4	5	May	1502588.82
5	6	June	1298162.91
6	7	July	1393538.70
7	8	August	1428658.01
8	9	September	624814.05
9	10	October	713727.09
10	11	November	1010271.37
11	12	December	743925.07

```
In [36]: # Which month generated the highest revenue?
con.sql("""
select
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    round(sum(oi.price),2) as total_revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    month,
    month_name
order by
    total_revenue desc
limit 1;
""").show()
```

month	month_name	total_revenue
int64	varchar	double
5	May	1502588.82

```
In [37]: # Which month generated the lowest revenue?
con.sql("""
select
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    round(sum(oi.price),2) as total_revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    month,
    month_name
order by
    total_revenue
limit 1;
""").show()
```

month	month_name	total_revenue
int64	varchar	double
9	September	624814.05

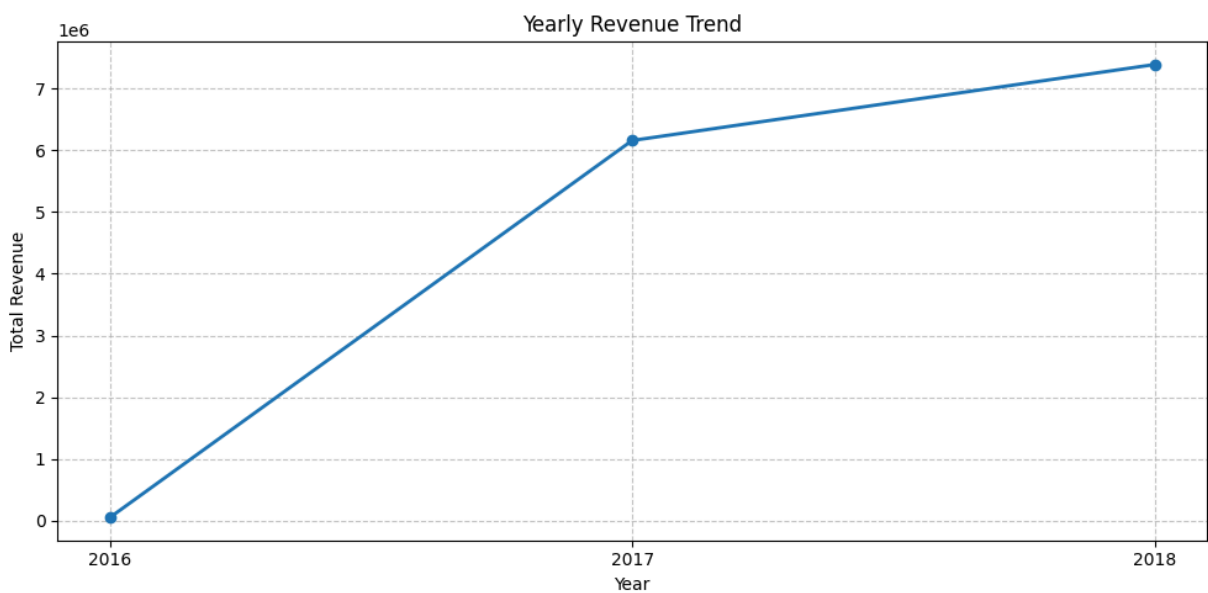
```
In [38]: # Is revenue growing over time?
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
```

```
plt.plot(
    yearly_revenue["year"],
    yearly_revenue["total_revenue"],
    marker='o',
    linewidth=2
)

plt.xticks(yearly_revenue['year'])
plt.title("Yearly Revenue Trend")
plt.xlabel("Year")
plt.ylabel("Total Revenue")
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

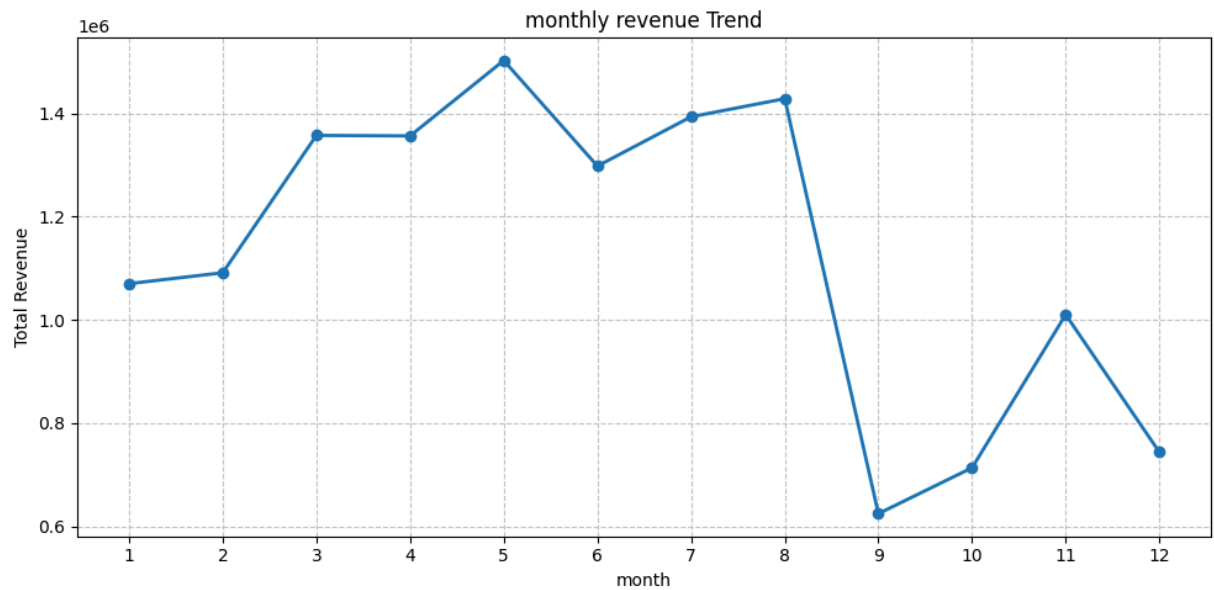


```
In [39]: plt.figure(figsize=(10, 5))

plt.plot(
    mnth_rev["month"],
    mnth_rev["total_revenue"],
    marker='o',
    linewidth=2
)

plt.xticks(mnth_rev['month'])
plt.title("monthly revenue Trend")
plt.xlabel("month")
plt.ylabel("Total Revenue")
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```



```
In [40]: # What is the Month-over-Month (MoM) revenue growth?
con.sql("""
with year_month_rev as(
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    sum(oi.price) as revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    year,
    month,
    month_name
order by
    year,
    month,
    revenue),

rev_grwth as(
select
    year,
    month_name,
    revenue,
    lag(revenue) over(
        order by year, month
    ) as prev_mnth_rev
from
    year_month_rev)

select
    year,
    month_name,
    revenue,
    prev_mnth_rev,
```

```
    ((revenue - prev_mnth_rev)*100/prev_mnth_rev) as mom_grwth_per  
from  
    rev_grwth;  
""").show()
```

year	month_name	revenue	prev_mnth_rev	mom_grwth_per
int64	varchar	double	double	double
2016	September	267.36	NULL	NU
2016	October	49507.66000000016	267.36	18417.2277079593
2016	December	10.9	49507.66000000016	-99.977983205023
2017	January	120312.86999999953	10.9	1103687.79816513
2017	February	247303.019999999545	120312.86999999953	105.549929945147
2017	March	374344.30000000086	247303.019999999545	51.37069494744
2017	April	359927.23000000015	374344.30000000086	-3.85128610212559
2017	May	506071.14000001014	359927.23000000015	40.603738150072
2017	June	433038.60000000504	506071.14000001014	-14.4312793651903
2017	July	498031.48000001034	433038.60000000504	15.0085650563262
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
2017	December	743914.1700000202	1010271.3700000371	-26.364916190786
2018	January	950030.3600000378	743914.1700000202	27.706985336764
2018	February	844178.7100000294	950030.3600000378	-11.1419228749704
2018	March	983213.4400000331	844178.7100000294	16.4698218935181
2018	April	996647.7500000333	983213.4400000331	1.3663676118991
2018	May	996517.6800000271	996647.7500000333	-0.0130507493752161
2018	June	865124.3100000215	996517.6800000271	-13.185252267676
2018	July	895507.2200000229	865124.3100000215	3.51197043578635
2018	August	854686.3300000255	895507.2200000229	-4.55840992549411
2018	September	145.0	854686.3300000255	-99.983034711693

24 rows (20 shown)
ns

5 colum

```
In [41]: # Which month recorded the highest growth?
con.sql("""
with year_month_rev as(
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    sum(oi.price) as revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    year,
    month,
    month_name
order by
    year,
    month,
    revenue),

rev_grwth as(
select
    year,
    month_name,
    revenue,
    lag(revenue) over(
        order by year, month
    ) as prev_mnth_rev
from
    year_month_rev)

select
    month_name,
    ((revenue - prev_mnth_rev)*100/prev_mnth_rev) as mom_grwth_per
from
    rev_grwth
order by mom_grwth_per desc
limit 1;

""").show()
```

month_name varchar	mom_grwth_per double
January	1103687.7981651332

```
In [42]: # Which month recorded the biggest decline?
con.sql("""
```

```

with year_month_rev as(
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    sum(oi.price) as revenue
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    year,
    month,
    month_name
order by
    year,
    month,
    revenue),

rev_grwth as(
select
    year,
    month_name,
    revenue,
    lag(revenue) over(
        order by year, month
    ) as prev_mnth_rev
from
    year_month_rev)

select
    month_name,
    ((revenue - prev_mnth_rev)*100/prev_mnth_rev) as mom_grwth_per
from
    rev_grwth
order by mom_grwth_per
limit 1;

""").show()

```

month_name varchar	mom_grwth_per double
September	-99.98303471169359

```

In [43]: # What is the cumulative (running) revenue over time?
con.sql("""
with year_month_rev as(
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month,
    monthname(o.order_purchase_timestamp) as month_name,
    sum(oi.price) as revenue

```

```
from
    orders o
join
    order_items oi on o.order_id = oi.order_id
group by
    year,
    month,
    month_name
order by
    year,
    month,
    revenue),

cum_grwth as(
select
    year,
    month_name,
    revenue,
    sum(revenue) over(
        order by year, month
    ) as cum_rev
from
    year_month_rev)

select
    month_name,
    cum_rev
from
    cum_grwth;

""").show()
```

month_name varchar	cum_rev double
September	267.36
October	49775.020000000164
December	49785.920000000166
January	170098.78999999997
February	417401.809999999517
March	791746.1099999996
April	1151673.33999999975
May	1657744.48000000077
June	2090783.08000000126
July	2588814.5600000023
.	.
.	.
.	.
December	6205592.9000000126
January	7155623.2600000164
February	7999801.9700000193
March	8983015.4100000227
April	9979663.160000026
May	10976180.8400000289
June	11841305.150000031
July	12736812.3700000333
August	13591498.7000000359
September	13591643.7000000359

24 rows (20 shown) 2 columns

In []:

```
In [44]: # Which product categories generate the highest revenue?
con.sql("""
select
    p.product_category_name,
    round(sum(oi.price),2) as revenue
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    revenue desc
limit 1;

""")
```

Out[44]:

product_category_name varchar	revenue double
beleza_saude	1258681.34

In [45]: *# Which product categories generate the lowest revenue?*

```
con.sql("""
select
    p.product_category_name,
    round(sum(oi.price),2) as revenue
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    revenue
limit 1;

""")
```

Out[45]:

product_category_name varchar	revenue double
seguros_e_servicos	283.29

In [46]: *#Top 10 categories by revenue?*

```
con.sql("""
select
    p.product_category_name,
    round(sum(oi.price),2) as revenue
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    revenue desc
limit 10;

""")
```

Out[46]:

product_category_name varchar	revenue double
beleza_saude	1258681.34
relogios_presentes	1205005.68
cama_mesa_banho	1036988.68
esporte_lazer	988048.97
informatica_acessorios	911954.32
moveis_decoracao	729762.49
cool_stuff	635290.85
utilidades_domesticas	632248.66
automotivo	592720.11
ferramentas_jardim	485256.46

10 rows

2 columns

In [47]: *#Bottom 10 categories by revenue?*

```
con.sql("""
select
    p.product_category_name,
    round(sum(oi.price),2) as revenue
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    revenue
limit 10;

""")
```

Out[47]:

product_category_name varchar	revenue double
seguros_e_servicos	283.29
fashion_roupa_infanto_juvenil	569.85
cds_dvds_musicais	730.0
casa_conforto_2	760.27
flores	1110.04
pc_gamer	1545.95
fraldas_higiene	1567.59
artes_e_artesanato	1814.01
la_cuisine	2054.99
fashion_esporte	2119.51

10 rows

2 columns

In [48]: *# What percentage of total revenue comes from the Top 10 categories?*

```
con.sql("""
with Top10_prdct_cat_rev as (
select
    p.product_category_name,
```

```

        sum(oi.price) as revenue
    from
        products p
    join
        order_items oi on p.product_id = oi.product_id
    group by
        p.product_category_name
    order by
        revenue desc
    limit 10)

select
    round((select sum(revenue) from Top10_prdct_cat_rev) * 100.0 /sum(oi.price),2)
    as top10_rev_per
from
    order_items oi;

""")

```

Out[48]:

top10_rev_per double
62.36

In [49]: *# Which category has the highest average selling price?*

```

con.sql("""
select
    p.product_category_name,
    round(avg(oi.price),2) as avg_sp
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    avg_sp desc
limit 1;
""").show()

```

product_category_name varchar	avg_sp double
pcs	1098.34

In [50]: *# Which category has the lowest average selling price?*

```

con.sql("""
select
    p.product_category_name,
    round(avg(oi.price),2) as avg_sp
from

```

```

    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    avg_sp
limit 1;
""").show()

```

product_category_name varchar	avg_sp double
casa_conforto_2	25.34

In [51]: *# Which categories receive the highest number of orders?*

```

con.sql("""
select
    p.product_category_name,
    count(distinct oi.order_id) as highest_oder_count
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    highest_oder_count desc
limit 1;
""").show()

```

product_category_name varchar	highest_oder_count int64
cama_mesa_banho	9417

```
In [52]: # Which categories receive the lowest number of orders?
con.sql("""
select
    p.product_category_name,
    count(distinct oi.order_id) as lowest_oder_count
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    lowest_oder_count
limit 1;
""").show()
```

product_category_name varchar	lowest_oder_count int64
seguros_e_servicos	2

```
In [53]: # Top 10 product categories in receiving orders
con.sql("""
select
    p.product_category_name,
    count(distinct oi.order_id) as cat_oder_count
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    cat_oder_count desc
limit 10;
""").show()
```

product_category_name varchar	cat_oder_count int64
cama_mesa_banho	9417
beleza_saude	8836
esporte_lazer	7720
informatica_acessorios	6689
moveis_decoracao	6449
utilidades_domesticas	5884
relogios_presentes	5624
telefonia	4199
automotivo	3897
brinquedos	3886

10 rows

2 columns

```
In [54]: # Which categories generate high revenue despite low order volume?
con.sql("""
with category_metrics as (
select
    p.product_category_name,
    sum(oi.price) as revenue_per_cat,
    count(distinct oi.order_id) as order_count
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name)

select
    product_category_name,
    order_count,
    revenue_per_cat,
    round((revenue_per_cat * 1.0/order_count),2) as avg_rev_per_ord
from
    category_metrics
order by
    avg_rev_per_ord desc
limit 10;

""").show()
```

product_category_name	order_count	revenue_per_cat
avg_rev_per_ord		
varchar	int64	double
pcs	181	222963.12999999998
portateis_casa_forno_e_cafe	75	47445.71000000001
eletrodomesticos_2	234	113317.73999999999
agro_industria_e_comercio	182	72530.47000000002
instrumentos_musicais	628	191498.87999999999
eletroportateis	630	190648.57999999935
portateis_cozinha_e_preparadores_de_alimentos	14	3968.53
telefonos_fixos	217	59582.99999999998
construcao_ferramentas_seguranca	167	40544.52000000005
climatizacao	253	55024.96000000008

10 rows
4 columns

```
In [55]: # Segment the products based on their value
con.sql("""
with category_metrics as (
select
    p.product_category_name,
    sum(oi.price) as cat_revenue,
    count(distinct oi.order_id) as order_count
from
    products p
join
    order_items oi on p.product_id = oi.product_id
group by
    p.product_category_name),

avg_rev_metrics as (
select
    product_category_name,
    order_count,
    cat_revenue,
    round((cat_revenue * 1.0/order_count),2) as avg_rev_per_ord
from
    category_metrics
```

```
order by
    avg_rev_per_ord desc)

select
    product_category_name,
    order_count as total_order,
    cat_revenue as total_revenue,
    avg_rev_per_ord,

    case
        when avg_rev_per_ord >= 500
            then 'Premium Products'
        when avg_rev_per_ord >= 300
            then 'High-Value Transactions'
        else 'Specialized Product Segments'
    end as "Category_Type"
from
    avg_rev_metrics
order by
    avg_rev_per_ord desc;

""").show()
```

product_category_name		total_order	total_revenue
avg_rev_per_ord	Category_Type		
double	varchar	int64	double
pcs		181	222963.12999999998
1231.84	Premium Products		
	portateis_casa_forno_e_cafe	75	47445.71000000001
632.61	Premium Products		
	eletrodomesticos_2	234	113317.73999999999
484.26	High-Value Transactions		
	agro_industria_e_comercio	182	72530.47000000002
398.52	High-Value Transactions		
	instrumentos_musicais	628	191498.8799999999
304.93	High-Value Transactions		
	eletroportateis	630	190648.57999999935
302.62	High-Value Transactions		
	portateis_cozinha_e_preparadores_de_alimentos	14	3968.53
283.47	Specialized Product Segments		
	telefonias_fixas	217	59582.99999999998
274.58	Specialized Product Segments		
	construcao_ferramentas_seguranca	167	40544.52000000005
242.78	Specialized Product Segments		
	climatizacao	253	55024.96000000008
217.49	Specialized Product Segments		
	.	.	.
.	.	.	.
	.	.	.
.	.	.	.
	.	.	.
.	.	.	.
	fashion_roupa_feminina	39	2803.640000000012
71.89	Specialized Product Segments		
	fashion_roupa_infanto_juvenil	8	569.85
71.23	Specialized Product Segments		
	artigos_de_natal	128	8800.81999999999
68.76	Specialized Product Segments		
	alimentos_bebidas	227	15179.47999999998
66.87	Specialized Product Segments		
	alimentos	450	29393.41000000016
65.32	Specialized Product Segments		
	eletronicos	2550	160246.73999999967
62.84	Specialized Product Segments		
	cds_dvds_musicais	12	730.0
60.83	Specialized Product Segments		
	fraldas_higiene	27	1567.590000000001
58.06	Specialized Product Segments		
	flores	29	1110.0399999999997
38.28	Specialized Product Segments		
	casa_conforto_2	24	760.2699999999998
31.68	Specialized Product Segments		

74 rows (20 shown)
5 columns

```
In [56]: # Which sellers generate the highest revenue?
con.sql("""
select
    seller_id,
    sum(price) as revenue
from
    order_items
group by
    seller_id
order by
    revenue desc
limit 1;
""").show()
```

seller_id varchar	revenue double
4869f7a5dfa277a7dca6462dcf3b52b2	229472.6299999981

```
In [57]: # Which sellers generate the lowest revenue?
con.sql("""
select
    seller_id,
    sum(price) as revenue
from
    order_items
group by
    seller_id
order by
    revenue
limit 1;
""").show()
```

seller_id varchar	revenue double
cf6f6bc4df3999b9c6440f124fb2f687	3.5

```
In [58]: # Top 10 sellers by revenue.
con.sql("""
select
    seller_id,
    sum(price) as revenue
from
    order_items
group by
    seller_id
```

```
order by
    revenue desc
limit 10;
""").show()
```

seller_id varchar	revenue double
4869f7a5dfa277a7dca6462dcf3b52b2	229472.62999999981
53243585a1d6dc2643021fd1853d8905	222776.04999999952
4a3ca9315b744ce9f8e9374361493884	200472.91999999949
fa1c13f2614d7b5c4749cbc52fecda94	194042.029999999846
7c67e1448b00f6e969d365cea6b010ab	187923.88999999995
7e93a43ef30c4f03f38b393420bc753a	176431.86999999982
da8622b14eb17ae2831f4ac5b9dab84a	160236.569999999538
7a67c85e85bb2ce8582c35f2203ad736	141745.530000000177
1025f0e2d44d7041d6cf58b6550e0bfa	138968.54999999995
955fee9216a65b617aa5c0531780ce60	135171.69999999914

10 rows

2 columns

```
In [59]: # Bottom 10 sellers by revenue.
con.sql("""
select
    seller_id,
    sum(price) as revenue
from
    order_items
group by
    seller_id
order by
    revenue
limit 10;
""").show()
```

seller_id varchar	revenue double
cf6f6bc4df3999b9c6440f124fb2f687	3.5
77128dec4bec4878c37ab7d6169d6f26	6.5
1fa2d3def6adfa70e58c276bb64fe5bb	6.9
702835e4b785b67a084280efca355756	7.6
34aefe746cd81b7f3b23253ea28bef39	8.0
ad14615bdd492b01b0d97922e87cb87f	8.25
4965a7002cca77301c82d3f91b82e1a9	8.49
0f94588695d71662beec8d883ffacf09	9.0
c18309219e789960add0b2255ca4b091	9.9
95cca791657aabeff15a07eb152d7841	9.99

10 rows

2 columns

```
In [60]: # What is the average revenue per seller?
con.sql("""
```

```

select
    seller_id,
    avg(price) as avg_rev_per_seller
from
    order_items
group by
    seller_id
order by
    avg_rev_per_seller desc
limit 10;

""").show()

```

seller_id varchar	avg_rev_per_seller double
80ceebb4ee9b31afb6c6a916a574a1e2	6729.0
ee27a8f15b1dded4d213a468ba4eb391	6499.0
585175ec331ea177fa47199e39a6170a	3549.0
abe021b01ba992245271b9aa422032df	3360.0
a00824eb9093d40e589b940ec45c4eb0	3133.3233333333333
e2a1ac9bf33e5549a2a4f834e70df2f8	2999.89
e908c0f3646e8b60375734a350d95d71	2951.0
d63c73efd41eb002280e7ec831424edb	2799.0
1444c08e64d55fb3c25f0f09c07ffc2	2749.0
039e6ad9dae79614493083e241147386	2585.0

10 rows

2 columns

```

In [61]: # What percentage of total revenue comes from the Top 10 sellers?
con.sql("""
with sel_rev_metrics as (
select
    seller_id,
    sum(price) as rev_per_seller
from
    order_items
group by
    seller_id
order by
    rev_per_seller desc
limit 10)

select
    (round((sum(rev_per_seller) * 100.0/(select sum(price) from order_items)),2)) as
from
    sel_rev_metrics;

""").show()

```

top_10_seller_rev_per double
13.15

```
In [62]: # What percentage of total revenue comes from the Top 20 sellers?
con.sql("""
with sel_rev_metrics as (
select
    seller_id,
    sum(price) as rev_per_seller
from
    order_items
group by
    seller_id
order by
    rev_per_seller desc
limit 20)

select
    (round((sum(rev_per_seller) * 100.0/(select sum(price) from order_items)),2)) as
from
    sel_rev_metrics;

""").show()
```

top_10_seller_rev_per double
21.09

Is seller revenue concentrated among a few sellers? Revenue is not highly concentrated among a few sellers. The marketplace has a diversified seller ecosystem, with revenue distributed across a large number of sellers. The business is therefore less dependent on any single seller or small group of sellers.

```
In [63]: # Which sellers process the highest number of orders?
con.sql("""
select
    seller_id,
    count(distinct order_id) as highest_no_of_orders
from
    order_items
group by
    seller_id
order by
    highest_no_of_orders desc
limit 1;
""").show()
```

seller_id varchar	highest_no_of_orders int64
6560211a19b47992c3666cc44a7e94c0	1854

```
In [64]: # Which sellers process the lowest number of orders?
con.sql("""
select
    seller_id,
    count(distinct order_id) as lowest_no_of_orders
from
    order_items
group by
    seller_id
order by
    lowest_no_of_orders
limit 1;
""").show()
```

seller_id varchar	lowest_no_of_orders int64
aa2640d851ec322526f1295bb20092	1

```
In [65]: # Top 10 sellers by order count.
con.sql("""
select
    seller_id,
    count(distinct order_id) as no_of_orders
from
    order_items
group by
    seller_id
order by
    no_of_orders desc
limit 10;
""").show()
```

seller_id varchar	no_of_orders int64
6560211a19b47992c3666cc44a7e94c0	1854
4a3ca9315b744ce9f8e9374361493884	1806
cc419e0650a3c5ba77189a1882b7556a	1706
1f50f920176fa81dab994f9023523100	1404
da8622b14eb17ae2831f4ac5b9dab84a	1314
955fee9216a65b617aa5c0531780ce60	1287
7a67c85e85bb2ce8582c35f2203ad736	1160
ea8482cd71df3c1969d7b9473ff13abc	1146
4869f7a5dfa277a7dca6462dcf3b52b2	1132
3d871de0142ce09b7081e2b9d1733cb1	1080

10 rows

2 columns

```
In [66]: # Average Revenue per Order for each Seller
con.sql("""
select
    seller_id,
    round((sum(price)*1.0/count(distinct order_id)),2) as avg_ord_per_slr
from
    order_items
group by
    seller_id
""").show()
```

seller_id varchar	avg_ord_per_slr double
48436dade18ac8b2bce089ec2a041202	88.93
9d7a1d34a5052409006425275ba1c2b4	81.14
7040e82f899a04d1b434b795a43b4617	45.93
a416b6a846a11724393025641d4edd5e	158.46
c864036feaab8c1659f65ea4faebe1da	180.24
41b39e28db005d9731d9d485a83b4c38	87.76
77530e9772f57a62c906e1c21538ab82	145.66
92eb0f42c21942b6552362b9b114707d	19.2
7a67c85e85bb2ce8582c35f2203ad736	122.19
ffc470761de7d0232558ba5e786e57b7	58.89
.	.
.	.
.	.
d4f91e8be86b38da2ee35bc489645a9d	49.99
dd264199fc8b687ad029de7de6d760e6	87.9
4cc43a222a0f94910728b62b97a8c5f8	108.5
8ec76bb0965af3f007692b26fa9d6623	1999.0
623ad9d0c1e14bb5464a8c72bff7b4af	95.0
15ac3c501e2599e4917316fde5c5669a	144.2
f5b84683a9bf9e1df748cf40f601b39c	83.9
b57e8460909fa137df7951b4a3b5ea84	198.0
7a188843479798ca813108fa7b959286	149.7
a1bea9970d8f65b240555f99c2214f7f	75.0

3095 rows (20 shown)

2 columns

In [67]: *# Average order per seller*

```
con.sql("""
select
    round((count(distinct order_id)*1.0/count(distinct seller_id)),2) as avg_ord_pe
from
    order_items
""").show()
```

avg_ord_per_slr double
31.88

In [68]: *# Do the highest-order sellers also generate the highest revenue?*

```
con.sql("""
select
    seller_id,
    count(distinct order_id) as no_of_orders,
    sum(price) as revenue,
    dense_rank() over(order by no_of_orders desc) as order_rank,
    dense_rank() over(order by revenue desc) as rev_rank
""")
```

```
from
    order_items
group by
    seller_id
order by
    no_of_orders desc;
""").show()
```

rev_rank	seller_id	no_of_orders	revenue	order_rank
int64	varchar	int64	double	int64
6560211a19b47992c3666cc44a7e94c012	4a3ca9315b744ce9f8e93743614938843	1854	123304.830000000003	1
cc419e0650a3c5ba77189a1882b7556a17	1f50f920176fa81dab994f902352310016	1806	200472.91999999949	2
da8622b14eb17ae2831f4ac5b9dab84a7	955fee9216a65b617aa5c0531780ce6010	1706	104288.42000000263	3
7a67c85e85bb2ce8582c35f2203ad7368	1f50f920176fa81dab994f902352310016	1404	106939.20999999739	4
ea8482cd71df3c1969d7b9473ff13abc62	da8622b14eb17ae2831f4ac5b9dab84a7	1314	160236.56999999538	5
4869f7a5dfa277a7dca6462dcf3b52b21	955fee9216a65b617aa5c0531780ce6010	1287	135171.69999999914	6
3d871de0142ce09b7081e2b9d1733cb119	7a67c85e85bb2ce8582c35f2203ad7368	1160	141745.53000000177	7
.	ea8482cd71df3c1969d7b9473ff13abc62	1146	37177.52000000046	8
.	4869f7a5dfa277a7dca6462dcf3b52b21	1132	229472.6299999981	9
.	3d871de0142ce09b7081e2b9d1733cb119	1080	94914.19999999965	10
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
aec99834ab5d38ead22766ada8497e871862	6c6cc59a5c00cacc5054ca06d65cc2821354	1	459.0	251
1b4b28463457a256e9a784ebe2a8f6301657	cb810bd318f51b3dc3c753944d61a1fa1078	1	1107.0	251
04843805947f0fc584fc1969b6e50fe71207	feb793c88d836c3a75efa4b5a3465d701855	1	699.42	251
7dc9a3355bae96dd82008b313d3a456c1479	8ec76bb0965af3f007692b26fa9d66231019	1	1750.0	251
5d043cd5512d4bd2f88e5ccdd5736c381689	cd2169d9f31ff6fbb9ca3373b0c9e3541755	1	1400.0	251
		1	466.0	251
		1	899.0	251
		1	1999.0	251
		1	650.0	251
		1	569.0	251

3095 rows (20 shown)
5 columns

In [69]: *# Which sellers have high revenue but low order volume?*

```
con.sql("""
WITH seller_metrics AS (
    SELECT
        seller_id,
        COUNT(DISTINCT order_id) AS order_volume,
        SUM(price) AS revenue
    FROM order_items
    GROUP BY seller_id
),

seller_rankings AS (
    SELECT
        seller_id,
        order_volume,
        revenue,
        DENSE_RANK() OVER (
            ORDER BY revenue DESC
        ) AS revenue_rank,
        DENSE_RANK() OVER (
            ORDER BY order_volume DESC
        ) AS order_volume_rank
    FROM seller_metrics
)

SELECT
    seller_id,
    order_volume,
    ROUND(revenue,2) AS revenue,
    revenue_rank,
    order_volume_rank
FROM seller_rankings
WHERE revenue_rank <= 20
    AND order_volume_rank > 100
ORDER BY revenue_rank;

""").show()
```

	seller_id	order_volume	revenue	revenue_rank	order_
volume_rank	varchar	int64	double	int64	
int64					
	edb1ef5e36e0c8cd84eb3c9b003e486d	166	79284.55	20	
101					

```
In [70]: # Which states have the highest number of sellers?
con.sql("""
select
    seller_state,
    count(seller_id) as no_of_sellers
from
    sellers
group by
    seller_state
order by
    no_of_sellers desc
Limit 1;
""").show()
```

seller_state varchar	no_of_sellers int64
SP	1849

```
In [71]: # Which states have the Lowest number of sellers?
con.sql("""
select
    seller_state,
    count(seller_id) as no_of_sellers
from
    sellers
group by
    seller_state
order by
    no_of_sellers
Limit 1;
""").show()
```

seller_state varchar	no_of_sellers int64
MA	1

```
In [72]: # Which states generate the highest seller revenue?
con.sql("""
select
    s.seller_state,
    sum(oi.price) as highest_sel_rev
from
    sellers s
join
    order_items oi on s.seller_id = oi.seller_id
group by
    s.seller_state
order by
```

```
highest_sel_rev desc
limit 1;

""")
```

Out[72]:

seller_state varchar	highest_sel_rev double
SP	8753396.210007336

In [73]: *# Which states generate the lowest seller revenue?*

```
con.sql("""
select
    s.seller_state,
    sum(oi.price) as lowest_sel_rev
from
    sellers s
join
    order_items oi on s.seller_id = oi.seller_id
group by
    s.seller_state
order by
    lowest_sel_rev
limit 1;

""")
```

Out[73]:

seller_state varchar	lowest_sel_rev double
AC	267.0

In [74]: *# Average revenue per seller by state.*

```
con.sql("""
select
    s.seller_state,
    round((sum(oi.price) * 1.0/count(distinct oi.seller_id)),2) as avg_revenue_per_
from
    sellers s
join
    order_items oi on s.seller_id = oi.seller_id
group by
    s.seller_state
order by
    avg_revenue_per_seller desc;
""").show()
```

seller_state varchar	avg_revenue_per_seller double
MA	36408.95
BA	15029.56
PE	10165.98
RJ	4935.58
SP	4734.12
MT	4267.68
MG	4145.76
PR	3615.72
SC	3328.56
DF	3258.32
RS	2934.57
PB	2849.17
PI	2522.0
RO	2381.1
ES	2073.46
RN	1998.52
MS	1710.34
GO	1659.98
CE	1556.97
PA	1238.0
AM	1177.0
SE	803.1
AC	267.0

23 rows

2 columns

```
In [75]: # Which state has the highest-performing seller?

con.sql("""
select
    s.seller_state,
    round((sum(oi.price) * 1.0/count(distinct oi.seller_id)),2) as avg_revenue_per_
from
    sellers s
join
    order_items oi on s.seller_id = oi.seller_id
group by
    s.seller_state
order by
    avg_revenue_per_seller desc
limit 1;
""").show()
```

seller_state varchar	avg_revenue_per_seller double
MA	36408.95

```
In [76]: # Segment sellers into: High Performers; Medium Performers; Low Performers
con.sql("""
with rank_rvn as (
select
    seller_id,
    sum(price) as revenue,
    dense_rank() over(order by sum(price) desc) as seller_rank
from
    order_items
group by
    seller_id)

select
    seller_id,
    revenue,
    seller_rank,
    case
        when seller_rank < 100 then
            'High Performer'
        when seller_rank between 101 and 500 then
            'Medium Prformer'
        else
            'Low Performer'
        end as seller_cat
from
    rank_rvn
order by
    seller_rank;

""").show()
```

seller_id	revenue	seller_rank	seller_cat
varchar	double	int64	varchar
4869f7a5dfa277a7dca6462dcf3b52b2	229472.6299999981	1	High Performer
53243585a1d6dc2643021fd1853d8905	222776.04999999952	2	High Performer
4a3ca9315b744ce9f8e9374361493884	200472.91999999949	3	High Performer
fa1c13f2614d7b5c4749cbc52fecda94	194042.02999999846	4	High Performer
7c67e1448b00f6e969d365cea6b010ab	187923.8899999995	5	High Performer
7e93a43ef30c4f03f38b393420bc753a	176431.86999999982	6	High Performer
da8622b14eb17ae2831f4ac5b9dab84a	160236.56999999538	7	High Performer
7a67c85e85bb2ce8582c35f2203ad736	141745.53000000177	8	High Performer
1025f0e2d44d7041d6cf58b6550e0bfa	138968.5499999995	9	High Performer
955fee9216a65b617aa5c0531780ce60	135171.69999999914	10	High Performer
.	.	.	.
.	.	.	.
.	.	.	.
95cca791657aabeff15a07eb152d7841	9.99	2777	Low Performer
c18309219e789960add0b2255ca4b091	9.9	2778	Low Performer
0f94588695d71662beec8d883ffac09	9.0	2779	Low Performer
4965a7002cca77301c82d3f91b82e1a9	8.49	2780	Low Performer
ad14615bdd492b01b0d97922e87cb87f	8.25	2781	Low Performer
34aefe746cd81b7f3b23253ea28bef39	8.0	2782	Low Performer
702835e4b785b67a084280efca355756	7.6	2783	Low Performer
1fa2d3def6adfa70e58c276bb64fe5bb	6.9	2784	Low Performer
77128dec4bec4878c37ab7d6169d6f26	6.5	2785	Low Performer
cf6f6bc4df3999b9c6440f124fb2f687	3.5	2786	Low Performer

3095 rows (20 shown)
ns

4 colum

```
In [77]: # How many customers are repeat customers?
con.sql("""
select
    c.customer_unique_id,
    count(distinct o.order_id) as repeat_cnt
from
    customers c
join
    orders o on o.customer_id = c.customer_id
group by
    customer_unique_id
having
    repeat_cnt > 1
order by
    repeat_cnt desc;

""").show()
```

customer_unique_id varchar	repeat_cnt int64
8d50f5eadf50201ccdcdfb9e2ac8455	17
3e43e6105506432c953e165fb2acf44c	9
1b6c7548a2a1f9037c1fd3ddfed95f33	7
6469f99c1f9dfae7733b25662e7f1782	7
ca77025e7201e3b30c44b472ff346268	7
dc813062e0fc23409cd255f7f53c7074	6
63cfc61cee11cbe306bff5857d00bfe4	6
f0e310a6839dce9de1638e0fe5ab282a	6
12f5d6e1cbf93dafd9dcc19095df0b3d	6
47c1a3033b8b77b3ab6e109eb4d5fdf3	6
.	.
.	.
.	.
7ce7c20ac269f4642f1c7058ab375102	2
8e78f2e8305481c8b0f285670de4829b	2
a44b31b99d33ba6c128b857d28842ea8	2
6bbbed249de21777c6f7d5074e6c469e9	2
730e4b7ede33b484516435d39ab4dc49	2
35de25c73544f00e8adde226a4d1bdcc	2
a2d8125361b16f316df65f4603b88e23	2
eed42da7ee2f3ce26ab52758e63d855f	2
b197f4af65829575f2f333bd0e9fd204	2
45df9d71b80f651fcfd26763eebd027a	2

2997 rows (20 shown)

2 columns

```
In [78]: # What percentage of customers are repeat customers?

con.sql("""
```

```

with repeat_cust_cnt as (
select
    c.customer_unique_id,
    count(distinct o.order_id) as repeat_cnt
from
    customers c
join
    orders o on o.customer_id = c.customer_id
group by
    customer_unique_id
having
    COUNT(DISTINCT o.order_id) > 1
order by
    repeat_cnt desc)

select
    round((count(customer_unique_id) * 100.0/(select count(distinct customer_unique
from
    repeat_cust_cnt;

""").show()

```

repeat_cust_per double
3.12

```

In [79]: # How much revenue comes from repeat customers?
con.sql("""
with repeat_cust_cnt as (
select
    c.customer_unique_id,
    count(distinct o.order_id) as repeat_cnt,
    sum(oi.price) as revenue
from
    customers c
join
    orders o on o.customer_id = c.customer_id
join
    order_items oi on oi.order_id = o.order_id
group by
    customer_unique_id
having
    COUNT(DISTINCT o.order_id) > 1
order by
    repeat_cnt desc)

select
    round(sum(revenue),2) as repeat_cust_rev
from
    repeat_cust_cnt;

""").show()

```

repeat_cust_rev
double
763291.86

In [80]: *# What percentage of total revenue comes from repeat customers?*

```
con.sql("""
with repeat_cust_cnt as (
select
    c.customer_unique_id,
    count(distinct o.order_id) as repeat_cnt,
    sum(oi.price) as revenue
from
    customers c
join
    orders o on o.customer_id = c.customer_id
join
    order_items oi on oi.order_id = o.order_id
group by
    customer_unique_id
having
    COUNT(DISTINCT o.order_id) > 1
order by
    repeat_cnt desc),

repeat_customer_revenue as
(
select
    round(sum(revenue),2) as repeat_cust_rev
from
    repeat_cust_cnt
)

select
    round((repeat_cust_rev * 100.0/(select sum(price) from order_items)),2) as repe
from
    repeat_customer_revenue;

""").show()
```

repeat_cust_rev_per
double
5.62

In [81]: *# Do repeat customers spend more than one-time customers?*

```
con.sql("""
with customer_metrics as (
```

```

select
    c.customer_unique_id,
    count(distinct o.order_id) as order_count,
    sum(oi.price) as revenue
from
    customers c
join
    orders o on o.customer_id = c.customer_id
join
    order_items oi on oi.order_id = o.order_id
group by
    c.customer_unique_id)

select
    case
        when order_count = 1 then
            'One-Time Customer'
        else
            'Repeat Customer'
        end as
        Customer_Type,
    count(customer_unique_id),
    avg(revenue)
from
    customer_metrics
group by
    Customer_Type;

""").show()

```

Customer_Type varchar	count(customer_unique_id) int64	avg(revenue) double
Repeat Customer	2913	262.0294747682802
One-Time Customer	92507	138.6743904785499

```

In [82]: # Which seller generates the highest revenue in each state?
con.sql("""
with seller_state_revenue as (
select
    oi.seller_id,
    s.seller_state,
    sum(oi.price) as revenue
from
    sellers s
join
    order_items oi on oi.seller_id = s.seller_id
group by
    oi.seller_id,
    s.seller_state),

```

```
seller_state_ranking as(
select
    seller_id,
    seller_state,
    revenue,
    dense_rank() over(
        partition by seller_state
        order by revenue desc
    ) as seller_state_rev_rank
from
    seller_state_revenue)
select
    seller_id,
    seller_state,
    revenue,
    seller_state_rev_rank
from
    seller_state_ranking
where
    seller_state_rev_rank = 1;
""")
```

Out[82]:

	seller_id	seller_state	revenue	seller_st
ate_rev_rank	varchar	varchar	double	i
nt64				
1	06a2c3af7b3aee5d69171b0e14f0ee87	MA	36408.95000000018	
1	a1043bafd471dff536d0c462352beb48	MG	101901.16000000053	
1	de722cd6dad950a92b7d4f82673f8833	PE	55426.10000000033	
1	47efca563408aae19bb7206c2d969ea9	PI	2522.0	
1	ccc4bbb5f32a6ab2b7066a4130f114e3	PR	74004.62000000001	
1	46dc3b2cc0980fb8ec44634e21d2718e	RJ	128111.1900000011	
1	02d35243ea2e497335cd0f076b45675d	RN	5794.0	
1	4b39558c138930b9e5489c93b6df5fe3	SE	1096.7	
1	04308b1ee57b6625f47df1d56f00eedf	SC	60130.60000000009	
1	53243585a1d6dc2643021fd1853d8905	BA	222776.04999999952	
1	001cca7ae9ae17fb1caed9dfb1094831	ES	25080.03000000017	
1	a6bd7d1ccdac48c6b33b28596b7eb122	PB	14368.61	
1	327b89b872c14d1c0be7235ef4871685	AM	1177.0	
1	b1fecf4da1fa2689bccffa0121953643	MS	3669.64	
1	bbf9ad41dca6603e614efcdad7aab8c4	CE	7846.0	
1	3364a91ec4d56c98e44174de954b94f6	RO	3579.939999999996	
1	44073f8b7e41514de3b7815dd0237f4f	DF	18729.639999999992	
1	67225bff54a172ff67579aaf583efd78	PA	1238.0	
1	4be2e7f96b4fd749d52dff41f80e39dd	AC	267.0	
1	9d5a9018aee56acb367ba9c3f05d1d6a	GO	10292.309999999994	
1	87142160b41353c4e5fca2360caf6f92	RS	31095.98000000072	
1	2dee2ce60de9709b1a24083217181a1f	MT	7784.49999999997	
1	4869f7a5dfa277a7dca6462dcf3b52b2	SP	229472.6299999981	

23 rows
4 columns

In [83]: *# Top 3 Sellers in Each State*

```
con.sql("""
with seller_state_revenue as (
select
    oi.seller_id,
    s.seller_state,
    sum(oi.price) as revenue
from
    sellers s
join
    order_items oi on oi.seller_id = s.seller_id
group by
    oi.seller_id,
    s.seller_state),

seller_state_ranking as(
select
    seller_id,
    seller_state,
    revenue,
    dense_rank() over(
        partition by seller_state
        order by revenue desc
    ) as seller_state_rev_rank
from
    seller_state_revenue)
select
    seller_id,
    seller_state,
    revenue,
    seller_state_rev_rank
from
    seller_state_ranking
where
    seller_state_rev_rank <= 3;
""")
```

Out[83]:

	seller_id	seller_state	revenue	seller_state_rev_rank
	varchar	varchar	double	int64
1	44073f8b7e41514de3b7815dd0237f4f	DF	18729.639999999992	
2	f3b80352b986ab4d1057a4b724be19d0	DF	10505.099999999995	
3	3c487ae8f8d7542beff5788e2e0aea83	DF	9385.699999999995	
1	53243585a1d6dc2643021fd1853d8905	BA	222776.04999999952	
2	c72de06d72748d1a0dfb2125be43ba63	BA	17522.0	
3	75d34ebb1bd0bd7dde40dd507b8169c3	BA	15048.279999999986	
1	001cca7ae9ae17fb1caed9dfb1094831	ES	25080.030000000017	
2	33dd941c27854f7625b968cc6195a552	ES	6787.5	
3	15aac934c58d886785ac1b17953ea898	ES	4353.9	
1	a6bd7d1ccdac48c6b33b28596b7eb122	PB	14368.61	
.	.	.	.	
.	.	.	.	
.	.	.	.	
3	b410bdd36d5db7a65dcd42b7ead933b8	GO	7726.9	
1	87142160b41353c4e5fca2360caf6f92	RS	31095.980000000072	
2	b32be1695eb7ec5f10f72d9610a12527	RS	26874.0	
3	12b9676b00f60f3b700e83af21824c0e	RS	26774.0	
1	2dee2ce60de9709b1a24083217181a1f	MT	7784.499999999997	
2	abcd2cb37d46c2c8fb1bf071c859fc5b	MT	4880.220000000001	
3	99002261c568a84cce14d43fcffb43ea	MT	4088.0	
1	4869f7a5dfa277a7dca6462dcf3b52b2	SP	229472.62999999981	
2	4a3ca9315b744ce9f8e9374361493884	SP	200472.91999999949	
3	fa1c13f2614d7b5c4749cbc52fecda94	SP	194042.029999999846	

57 rows (20 shown)
4 columns

```
In [84]: # Top Product Category in Each Month
# Month - customers, Product Category - products, Top - revenue needed - order_items

con.sql("""
with mix_cust_matrices as (
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month_fig,
    monthname(o.order_purchase_timestamp) as month,
    p.product_category_name,
    sum(oi.price) as revenue
from
    customers c
join
    orders o on o.customer_id = c.customer_id
join
    order_items oi on oi.order_id = o.order_id
join
    products p on p.product_id = oi.product_id
group by
    year,
    month_fig,
    month,
    p.product_category_name
order by
    month_fig,
    product_category_name),

ranking_prod_month as (
select
    year,
    month_fig,
    month,
    product_category_name,
    revenue,
    dense_rank() over(partition by year, month_fig order by revenue desc) as prod_cat_ranking
from
    mix_cust_matrices)

select
    year,
    month,
    product_category_name,
    prod_cat_ranking,
    revenue
from
    ranking_prod_month
where
    prod_cat_ranking = 1
order by
    year,
    month_fig;
""")
```

Out[84]:

	year	month	product_category_name	prod_cat_ra
ranking	revenue			
int64	varchar		varchar	int64
	double			
1	2016	September	beleza_saude	
		134.97		
1	2016	October	moveis_decoracao	
		5807.889999999999		
1	2016	December	fashion_bolsas_e_acessorios	
		10.9		
1	2017	January	moveis_decoracao	
		13521.309999999997		
1	2017	February	beleza_saude	
		22838.790000000045		
1	2017	March	informatica_acessorios	
		28624.600000000046		
1	2017	April	esporte_lazer	
		24864.330000000038		
1	2017	May	beleza_saude	
		46786.020000000084		
1	2017	June	cool_stuff	
		38244.500000000005		
1	2017	July	cama_mesa_banho	
		63888.75000000026		
	.	.	.	
	.	.	.	
	.	.	.	
	.	.	.	
	.	.	.	
1	2017	December	relogios_presentes	
		71727.620000000005		
1	2018	January	esporte_lazer	
		91574.329999999993		
1	2018	February	informatica_acessorios	
		100855.229999999972		
1	2018	March	relogios_presentes	
		97861.079999999996		
1	2018	April	relogios_presentes	
		92658.569999999989		
1	2018	May	relogios_presentes	
		123872.659999999955		
1	2018	June	beleza_saude	
		107908.820000000008		
1	2018	July	beleza_saude	
		105813.030000000001		
1	2018	August	beleza_saude	
		120803.939999999973		
1	2018	September	moveis_cozinha_area_de_servico_jantar_e_jardim	
		145.0		

24 rows (20 shown)
5 columns

```
In [85]: # Top 3 Product Categories in Each Month
# Top Product Category in Each Month
# Month - customers, Product Category - products, Top - revenue needed - order_items

con.sql("""
with mix_cust_matrices as (
select
    year(o.order_purchase_timestamp) as year,
    month(o.order_purchase_timestamp) as month_fig,
    monthname(o.order_purchase_timestamp) as month,
    p.product_category_name,
    sum(oi.price) as revenue
from
    customers c
join
    orders o on o.customer_id = c.customer_id
join
    order_items oi on oi.order_id = o.order_id
join
    products p on p.product_id = oi.product_id
group by
    year,
    month_fig,
    month,
    p.product_category_name
order by
    month_fig,
    product_category_name),

ranking_prod_month as (
select
    year,
    month_fig,
    month,
    product_category_name,
    revenue,
    dense_rank() over(partition by year, month_fig order by revenue desc) as prod_cat_ranking
from
    mix_cust_matrices)

select
    year,
    month,
    product_category_name,
    prod_cat_ranking,
    revenue
from
    ranking_prod_month
where
    prod_cat_ranking <=3
order by
    year,
    month_fig,
```

```
revenue desc;  
""")
```

Out[85]:

	year	month	product_category_name	prod_cat_ra
ranking	revenue			
int64	varchar		varchar	int64
	double			
1	2016	September	beleza_saude	
		134.97		
2	2016	September	moveis_decoracao	
		72.89		
3	2016	September	telefonica	
		59.5		
1	2016	October	moveis_decoracao	
		5807.889999999999		
2	2016	October	perfumaria	
		5688.699999999999		
3	2016	October	beleza_saude	
		4552.509999999999		
1	2016	December	fashion_bolsas_e_acessorios	
		10.9		
1	2017	January	moveis_decoracao	
		13521.309999999997		
2	2017	January	beleza_saude	
		12561.319999999987		
3	2017	January	esporte_lazer	
		9717.619999999994		
	.	.	.	
	.	.	.	
	.	.	.	
	.	.	.	
	.	.	.	
1	2018	June	beleza_saude	
		107908.82000000008		
2	2018	June	relogios_presentes	
		86885.559999999991		
3	2018	June	cama_mesa_banho	
		71565.48000000008		
1	2018	July	beleza_saude	
		105813.03000000001		
2	2018	July	relogios_presentes	
		96212.589999999988		
3	2018	July	utilidades_domesticas	
		63409.140000000094		
1	2018	August	beleza_saude	
		120803.939999999973		
2	2018	August	relogios_presentes	
		72277.06000000003		
3	2018	August	cama_mesa_banho	
		61035.880000000012		
1	2018	September	moveis_cozinha_area_de_servico_jantar_e_jardim	
		145.0		

68 rows (20 shown)
5 columns

```
In [86]: # What percentage of total revenue is contributed by each product category?
con.sql("""
select
    p.product_category_name,
    sum(oi.price) as revenue,
    (sum(oi.price) * 100.0/(select sum(price) from order_items)) as rev_per
from
    order_items oi
join
    products p on p.product_id = oi.product_id
group by
    p.product_category_name
order by
    rev_per desc;
""")
```

Out[86]:

product_category_name varchar	revenue double	rev_per double
beleza_saude	1258681.3399999682	9.260699940204097
relogios_presentes	1205005.6799999995	8.865783319487262
cama_mesa_banho	1036988.6800000712	7.629604651856703
esporte_lazer	988048.9700000401	7.269532602587342
informatica_acessorios	911954.3200000388	6.709669118232453
moveis_decoracao	729762.4900000411	5.369199679647865
cool_stuff	635290.8500000009	4.674128192452084
utilidades_domesticas	632248.6600000213	4.6517453955871515
automotivo	592720.1100000107	4.360915596981047
ferramentas_jardim	485256.46000001475	3.5702558918573484
.	.	.
.	.	.
.	.	.
fashion_esporte	2119.510000000001	0.015594213965436633
la_cuisine	2054.99	0.015119510526882441
artes_e_artesanato	1814.0100000000002	0.01334650937029865
fraldas_higiene	1567.5900000000001	0.011533483621251514
pc_gamer	1545.95	0.011374268146820138
flores	1110.0399999999997	0.008167076951839465
casa_conforto_2	760.2699999999998	0.005593657520607356
cds_dvds_musicais	730.0	0.005370947150411527
fashion_roupa_infanto_juvenil	569.85	0.0041926496351534365
seguros_e_servicos	283.28999999999996	0.0020842953674521662

74 rows (20 shown)

3 columns

In [108... *# What percentage of categories contribute 80% of total revenue?*

```
con.sql("""
with revenue_per as (
select
    p.product_category_name,
    sum(oi.price) as revenue
```

```

from
    order_items oi
join
    products p on p.product_id = oi.product_id
group by
    p.product_category_name),

cumulative_revenue as (
select
    product_category_name,
    sum(revenue) over(order by revenue desc, product_category_name) as cum_rev
from
    revenue_per),

top_products as (
select
    product_category_name,
    round((cum_rev*100.0/(select sum(price) from order_items)),2) as prod_rev_per
from
    cumulative_revenue)

select
    count(*) as Top_Prod_Cat_Count,
    (select count(distinct product_category_name) from products) as Total_Prod_Cat_Count,
    round((count(*)*100.0/(select count(distinct product_category_name) from products)) as prod_cat_per
from
    top_products
where
    prod_rev_per <= 81;

""")

```

Out[108...

Top_Prod_Cat_Count int64	Total_Prod_Cat_Count int64	prod_cat_per double
17	73	23.29

In [122...

```

# What percentage of sellers contribute 80% of total revenue?
con.sql("""
with sellers_revenue as (
select
    seller_id,
    sum(price) as revenue
from
    order_items
group by
    seller_id
order by
    revenue desc),

seller_running_rev as (
select
    seller_id,
    revenue,

```

```

        sum(revenue) over(order by revenue desc, seller_id) as sel_cum_rev
from
    sellers_revenue
order by
    revenue desc),

seller_revenue_per as (
select
    seller_id,
    revenue,
    sel_cum_rev,
    (sel_cum_rev * 100.0/(select sum(price) from order_items)) as sel_cum_rev_per
from
    seller_running_rev
order by
    revenue desc),

Top_Sellers as (
select
    seller_id,
    revenue,
    sel_cum_rev,
    sel_cum_rev_per
from
    seller_revenue_per
where
    sel_cum_rev_per <= 80
order by
    revenue desc)

select
    count(*) as Top_Sellers_Count,
    (select count(distinct seller_id) from order_items) as Total_Seller_Count,
    round((Top_Sellers_Count * 100.0/(select count(distinct seller_id) from order_i
from
    Top_Sellers;

""").show()

```

Top_Sellers_Count int64	Total_Seller_Count int64	top_sel_per double
543	3095	17.54

In [132...

```

# Rank all sellers by revenue within their own state.
con.sql("""
with seller_state_rev as (
select
    o.seller_id,
    s.seller_state,
    sum(o.price) as revenue
from
    order_items o

```

```
join
    sellers s on s.seller_id = o.seller_id
group by
    o.seller_id,
    s.seller_state)

select
    seller_id,
    seller_state,
    revenue,
    dense_rank() over(partition by seller_state order by revenue desc) as seller_st
from
    seller_state_rev
order by
    seller_state,
    revenue desc;

""")
```

Out[132...

ate_rank	seller_id	seller_state	revenue	seller_st
64	varchar	varchar	double	int
1	4be2e7f96b4fd749d52dff41f80e39dd	AC	267.0	
1	327b89b872c14d1c0be7235ef4871685	AM	1177.0	
1	53243585a1d6dc2643021fd1853d8905	BA	222776.049999999952	
2	c72de06d72748d1a0dfb2125be43ba63	BA	17522.0	
3	75d34ebb1bd0bd7dde40dd507b8169c3	BA	15048.2799999999986	
4	d03698c2efd04a549382afa6623e27fb	BA	8865.4700000000001	
5	4aba391bc3b88717ce08eb11e44937b2	BA	7595.87	
6	a3dd39f583bc80bd8c5901c95878921e	BA	4607.80999999999995	
7	1444c08e64d55fb3c25f0f09c07ffcf2	BA	2749.0	
8	659e8466eb3ff1b0e8740d74fb7bbedd	BA	1486.8	
.	.	.	.	
.	.	.	.	
.	.	.	.	
.	.	.	.	
1720	c1dde11f12d05c478f5de2d7319ad3b2	SP	12.5	
1720	9e25199f6ef7e7c347120ff175652c3b	SP	12.5	
1720	7ab0dd5487bab2dc835337b244f689fb	SP	12.5	
1721	3d62f86afa7c73be2628a3be1423f5a0	SP	12.0	
1722	3ac588cd562971392504a9e17130c40b	SP	11.9	
1722	cc1f04647be106ba74e62b21f358af25	SP	11.9	
1723	4965a7002cca77301c82d3f91b82e1a9	SP	8.49	
1724	1fa2d3def6adfa70e58c276bb64fe5bb	SP	6.9	
1725	77128dec4bec4878c37ab7d6169d6f26	SP	6.5	
1726	cf6f6bc4df3999b9c6440f124fb2f687	SP	3.5	

3095 rows (20 shown)
columns

4

```
In [137... # Segment customers into High Value, Medium Value, and Low Value customers based on
con.sql("""
with customer_ranking as (
select
    c.customer_unique_id,
    count(distinct o.order_id) as order_count,
    sum(oi.price) as revenue,
    ntile(10) over(order by sum(oi.price) desc) as customer_rank
from
    customers c
join
    orders o on c.customer_id = o.customer_id
join
    order_items oi on oi.order_id = o.order_id
group by
    c.customer_unique_id)

select
    customer_unique_id,
    order_count,
    revenue,
    customer_rank,
    case
        when customer_rank = 1 then
            'High Value'
        when customer_rank between 2 and 4 then
            'Medium Value'
        else
            'Low Value'
        end as Customer_Value
from
    customer_ranking
order by
    customer_rank;
""")
```

Out[137...

customer_unique_id	order_count	revenue	customer_rank	Custo
mer_Value				
varchar	int64	double	int64	va
rchard				
0a0a92112bd4c708ca5fde585afaa872	1	13440.0	1	High
Value				
da122df9eeddfedc1dc1f5349a1a690c	2	7388.0	1	High
Value				
763c8b1c9c68a0229c42c9fc6f662b93	1	7160.0	1	High
Value				
dc4802a71eae9be1dd28f5d788ceb526	1	6735.0	1	High
Value				
459bef486812aa25204be022145caa62	1	6729.0	1	High
Value				
ff4159b92c40ebe40454e3e6a7c35ed6	1	6499.0	1	High
Value				
4007669dec559734d6f53e029e360987	1	5934.6	1	High
Value				
eebb5dda148d3893cdaf5b5ca3040ccb	1	4690.0	1	High
Value				
5d0a2980b292d049061542014e8960bf	1	4599.9	1	High
Value				
48e1ac109decbb87765a3eade6854098	1	4590.0	1	High
Value				
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
305adb7f869f2d07fa2170b042abefdf	1	275.8	2	Mediu
m Value				
154a5a9962a76cee141236c96aa045ca	1	275.8	2	Mediu
m Value				
670df3711f72fcfcd719c07d8d2e568e	1	275.8	2	Mediu
m Value				
531f8d2a8f04b3fbbb76e35ceaf8dfe2	1	275.8	2	Mediu
m Value				
cfa2652966bbc756e9ea2beacfbf61cf	1	275.5	2	Mediu
m Value				
571a7cd567b546ab8cb4096afe2154e0	1	275.49	2	Mediu
m Value				
e22a012287183808c019a479edb7295a	1	275.49	2	Mediu
m Value				
09d74edf20acb4f9523fb1cf19a18456	2	275.42	2	Mediu
m Value				
0e3c753956e35026b965d9ac3590c5af	2	275.35	2	Mediu
m Value				
4e012ebff739bde041e02fcabd42d3d0	1	275.31	2	Mediu
m Value				

? rows (>9999 rows, 20 shown)
5 columns

In [146...

What percentage of total revenue comes from the Top 10 customers?

```
con.sql("""
with customer_ranking as (
select
    c.customer_unique_id,
    count(distinct o.order_id) as order_count,
    sum(oi.price) as revenue,
    dense_rank() over(order by sum(oi.price) desc) as customer_rank
from
    customers c
join
    orders o on c.customer_id = o.customer_id
join
    order_items oi on oi.order_id = o.order_id
group by
    c.customer_unique_id)

select
    COUNT(*) AS top_customer_count,
    ROUND(SUM(revenue),2) AS top_customer_revenue,
    round((sum(revenue)*100.0/(select sum(price) from order_items)),2) as top_cust_
from
    customer_ranking
where
    customer_rank < 11;
""")
```

Out[146...

top_customer_count int64	top_customer_revenue double	top_cust_rev_per double
10	67765.5	0.5

In [178...

```
con.sql("""
WITH monthly_revenue AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        MONTH(o.order_purchase_timestamp) AS month_fig,
        MONTHNAME(o.order_purchase_timestamp) AS month_name,
        p.product_category_name,
        SUM(oi.price) AS revenue
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    JOIN products p
        ON p.product_id = oi.product_id
    WHERE p.product_category_name IS NOT NULL
    GROUP BY
        YEAR(o.order_purchase_timestamp),
        MONTH(o.order_purchase_timestamp),
        MONTHNAME(o.order_purchase_timestamp),
```

```

        p.product_category_name
    ),

    months AS (
        SELECT DISTINCT
            YEAR(order_purchase_timestamp) AS year,
            MONTH(order_purchase_timestamp) AS month_fig,
            MONTHNAME(order_purchase_timestamp) AS month_name
        FROM orders
    ),

    categories AS (
        SELECT DISTINCT
            product_category_name
        FROM products
        WHERE product_category_name IS NOT NULL
    ),

    category_month_grid AS (
        SELECT
            m.year,
            m.month_fig,
            m.month_name,
            c.product_category_name
        FROM months m
        CROSS JOIN categories c
    ),

    category_month_revenue AS (
        SELECT
            g.year,
            g.month_fig,
            g.month_name,
            g.product_category_name,
            COALESCE(r.revenue,0) AS revenue
        FROM category_month_grid g
        LEFT JOIN monthly_revenue r
            ON g.year = r.year
            AND g.month_fig = r.month_fig
            AND g.product_category_name = r.product_category_name
    ),

    previous_month AS (
        SELECT
            year,
            month_fig,
            month_name,
            product_category_name,
            revenue,
            LAG(revenue) OVER(
                PARTITION BY product_category_name
                ORDER BY year, month_fig
            ) AS prev_month_revenue
        FROM category_month_revenue
    ),

```

```

growth_calc AS (
  SELECT
    year,
    month_fig,
    month_name,
    product_category_name,
    revenue,
    prev_month_revenue,

    ROUND(
      (
        (revenue - prev_month_revenue)
        * 100.0
      ) /
      NULLIF(prev_month_revenue,0),
      2
    ) AS mom_growth_pct

  FROM previous_month
),

growth_rank AS (
  SELECT
    *,
    DENSE_RANK() OVER(
      ORDER BY mom_growth_pct DESC
    ) AS growth_rank
  FROM growth_calc
  WHERE mom_growth_pct IS NOT NULL
)

SELECT
  year,
  month_name,
  product_category_name,
  revenue,
  prev_month_revenue,
  mom_growth_pct
FROM growth_rank
WHERE growth_rank = 1;
""").show()

```

year	month_name	product_category_name	revenue	prev_month_revenue
2017	January	fashion_bolsas_e_acessorios	1671.970000000001	15239.17

In [191...

Which categories appeared most frequently in the monthly Top 3 categories?

```
con.sql("""
WITH monthly_revenue AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        MONTH(o.order_purchase_timestamp) AS month_fig,
        MONTHNAME(o.order_purchase_timestamp) AS month,
        p.product_category_name,
        SUM(oi.price) AS revenue
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    JOIN products p
        ON p.product_id = oi.product_id
    WHERE p.product_category_name IS NOT NULL
    GROUP BY
        YEAR(o.order_purchase_timestamp),
        MONTH(o.order_purchase_timestamp),
        MONTHNAME(o.order_purchase_timestamp),
        p.product_category_name
),

prod_ranking as (
select
    year,
    month_fig,
    month,
    product_category_name,
    revenue,
    dense_rank() over(partition by year, month_fig order by revenue desc) as prod_m
from
    monthly_revenue
order by
    year,
    month_fig
),

mnthly_top_3_prod as (select
    year,
    month,
    product_category_name,
    revenue,
    prod_mnth_rank
from
    prod_ranking
where
    prod_mnth_rank <=3
order by
    year,
    month_fig,
    prod_mnth_rank
)
select
    product_category_name as most_frequent_prod,
    count(*) as appeared_count
```

```

from
    mnthly_top_3_prod
group by
    most_frequent_prod
order by
    appeared_count desc;
""").df()

```

Out[191...

	most_frequent_prod	appeared_count
0	beleza_saude	16
1	relogios_presentes	12
2	cama_mesa_banho	12
3	esporte_lazer	10
4	informatica_acessorios	7
5	moveis_decoracao	4
6	pcs	1
7	utilidades_domesticas	1
8	moveis_cozinha_area_de_servico_jantar_e_jardim	1
9	fashion_bolsas_e_acessorios	1
10	cool_stuff	1
11	perfumaria	1
12	telefonica	1

In [207...

```

# What percentage of each month's revenue is contributed by each category?
con.sql("""
WITH monthly_revenue AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        MONTH(o.order_purchase_timestamp) AS month_fig,
        MONTHNAME(o.order_purchase_timestamp) AS month,
        p.product_category_name,
        SUM(oi.price) AS revenue
    FROM orders o
    JOIN order_items oi
        ON oi.order_id = o.order_id
    JOIN products p
        ON p.product_id = oi.product_id
    WHERE p.product_category_name IS NOT NULL
    GROUP BY
        YEAR(o.order_purchase_timestamp),
        MONTH(o.order_purchase_timestamp),
        MONTHNAME(o.order_purchase_timestamp),
        p.product_category_name
),
monthly_prod_rev as (

```

```
select
    year,
    month_fig,
    month,
    product_category_name,
    revenue,
    sum(revenue) over(partition by year, month) as month_tot_rev
from
    monthly_revenue
order by
    year,
    month_fig,
    revenue desc
)
select
    year,
    month,
    product_category_name,
    revenue,
    month_tot_rev,
    round((revenue*100.0/month_tot_rev),2) as prod_mon_rev_per
from
    monthly_prod_rev
order by
    year,
    month_fig;

""")
```

Out[207...

year	month	product_category_name	revenue
int64	month_tot_rev	prod_mon_rev_per	double
varchar	double	double	
double			
2016	September	beleza_saude	
134.97	267.36	50.48	
2016	September	moveis_decoracao	
72.89	267.36	27.26	
2016	September	telefonica	
59.5	267.36	22.25	
2016	October	moveis_decoracao	5807.88999
9999995	49441.76999999999	11.75	
2016	October	perfumaria	5688.69999
9999998	49441.76999999999	11.51	
2016	October	beleza_saude	4552.50999
9999999	49441.76999999999	9.21	
2016	October	brinquedos	4465.08999
9999999	49441.76999999999	9.03	
2016	October	consoles_games	
3882.26	49441.76999999999	7.85	
2016	October	relogios_presentes	3360.240000
0000002	49441.76999999999	6.8	
2016	October	esporte_lazer	3333.640000
0000003	49441.76999999999	6.74	
.	.	.	
.	.	.	
.	.	.	
.	.	.	
.	.	.	
2018	August	musica	
497.81	852267.290000001	0.06	
2018	August	fashion_calcados	471.2999999
9999995	852267.290000001	0.06	
2018	August	fashion_roupa_masculina	381.6900000
0000005	852267.290000001	0.04	
2018	August	fraldas_higiene	
231.89	852267.290000001	0.03	
2018	August	fashion_underwear_e_moda_praia	
219.9	852267.290000001	0.03	
2018	August	pc_gamer	
219.0	852267.290000001	0.03	
2018	August	fashion_roupa_feminina	
129.97	852267.290000001	0.02	
2018	August	fashion_esporte	
69.9	852267.290000001	0.01	
2018	August	dvds_blu_ray	
8.9	852267.290000001	0.0	
2018	September	moveis_cozinha_area_de_servico_jantar_e_jardim	
145.0	145.0	100.0	

1262 rows (20 shown)
6 columns

In [220...

```
# RFM Analysis R= Recency Score, F=Frequency Score, M= Monetary Score
con.sql("""
with cust_rfm_base as (
select
    c.customer_unique_id,
    datediff('day', Max(o.order_purchase_timestamp) , (select Max(order_purchase_ti
count(distinct o.order_id) as frequency,
    sum(price) as monetary
from
    customers c
join
    orders o on c.customer_id = o.customer_id
join
    order_items oi on o.order_id = oi.order_id
group by
    c.customer_unique_id
order by
    recency,
    frequency desc,
    monetary desc
),
r_f_m_score as(
select
    customer_unique_id,
    recency,
    frequency,
    monetary,
    6 - ntile(5) over(order by recency) as r_score,
    6 - ntile(5) over(order by frequency desc) as f_score,
    6 - ntile(5) over(order by monetary desc) as m_score
from
    cust_rfm_base)

select
    customer_unique_id,
    recency,
    frequency,
    monetary,
    r_score,
    f_score,
    m_score,
    concat(r_score,f_score,m_score) as rfm_score,
    CASE
    WHEN r_score >= 4
        AND f_score >= 4
        AND m_score >= 4
    THEN 'Champion'

    WHEN r_score >= 3
        AND f_score >= 3
        AND m_score >= 3
    THEN 'Loyal Customer'
```

```
        WHEN r_score >= 4
            AND f_score <= 2
        THEN 'Potential Loyalist'

        WHEN r_score <= 2
            AND f_score <= 2
            AND m_score <= 2
        THEN 'At Risk'

        ELSE 'Regular Customer'
    END AS customer_segment

from
    r_fm_score
order by
    rfm_score desc;

""")
```

Out[220...

score	customer_unique_id			recency	frequency	monetary	r_
nt64	f_score	r_score	rfm_score	customer_segment	int64	double	i
	varchar			int64	int64		
	int64	int64	varchar		varchar		
5	c8460e4251689ba205045f3ea17884a1	70	4	4080.0			
5	5	5	555	Champion			
5	906a8a4ec9f3d4c3e64fa6d1c4fe6009	89	2	1835.0			
5	5	5	555	Champion			
5	0341bbd5c969923a0f801b9e2d10a7b8	140	2	1828.44			
5	5	5	555	Champion			
5	4facc2e6fbc2bffb2fea92d2b4aa7e4	65	4	1686.9			
5	5	5	555	Champion			
5	525cda9909aa001ebed396f6e55eae01	71	2	1684.29			
5	5	5	555	Champion			
5	397b44d5bb99eabf54ea9c2b41ebb905	122	4	1650.0			
5	5	5	555	Champion			
5	fe81bb32c243a86b2f86fbf053fe6140	118	5	1535.4			
5	5	5	555	Champion			
5	fc24db02becd484accefaa5af59c18b1	114	2	1499.98			
5	5	5	555	Champion			
5	012a218df8995d3ec3bb221828360c86	121	2	1429.8000000000002			
5	5	5	555	Champion			
5	4711348768db55c57a67aba080ead566	132	2	1368.99			
5	5	5	555	Champion			
	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
5	83ccefd1c66a787e7bd1be2601b558e4	121	1	93.9			
5	3	5	533	Loyal Customer			
5	c298875962142d382e5035ca9f1cb428	121	1	93.9			
5	3	5	533	Loyal Customer			
5	66497ba44949e12429b9e0a1a4f867e2	123	1	93.9			
5	3	5	533	Loyal Customer			
5	e6af0fbd4d6e9c362378ecb52a67bbbd	124	1	93.9			
5	3	5	533	Loyal Customer			
5	27de9d99025697f85b5835d6c5e71cc4	124	1	93.9			
5	3	5	533	Loyal Customer			
5	6e8743c0e88d51e1e7b2455707e209e4	82	1	93.88999999999999			
5	3	5	533	Loyal Customer			
5	b28b906d2e7c601ca1311dbefff7962e	76	1	93.8			
5	3	5	533	Loyal Customer			
5	3e9c196f94da349ceb80fbef9c18e870	62	1	93.8			
5	3	5	533	Loyal Customer			
5	f6edf8b72c39e8ecec4f6b675ff642f1	91	1	93.8			
5	3	5	533	Loyal Customer			
5	141af08d486e290abbe5c61c8695f2cf	139	1	93.8			
5	3	5	533	Loyal Customer			

? rows (>9999 rows, 20 shown)
9 columns

E-Commerce Marketplace Analytics using SQL

Project Overview

Performed end-to-end business analytics on an e-commerce marketplace dataset containing approximately:

- 100K+ Orders
- 96K+ Customers
- 3K+ Sellers
- 70+ Product Categories

The objective of this project was to analyze customer behavior, seller performance, product performance, revenue trends, growth patterns, and marketplace dynamics using advanced SQL techniques to generate actionable business insights and recommendations.

Dataset Tables Used

- Customers
 - Orders
 - Order Items
 - Products
 - Sellers
 - Payments
-

Business Areas Covered

Customer Analytics

- Repeat vs One-Time Customers
- Customer Segmentation
- Customer Revenue Analysis
- RFM Analysis
- Customer Revenue Contribution

Seller Analytics

- Top Sellers
- Seller Revenue Contribution
- Seller Ranking by State
- Seller Pareto Analysis

Product Analytics

- Top Product Categories
- Category Revenue Contribution
- Top Categories by Month
- Consistently Top Performing Categories

Revenue Analytics

- Revenue Trends
- Running Revenue Analysis
- Revenue Concentration Analysis
- Revenue Leakage Analysis

Time-Series Analytics

- Month-over-Month Revenue Analysis
 - Monthly Revenue Contribution Analysis
 - Fastest Growing Categories
-

Business Questions Solved

Data Quality Assessment

- Are there duplicate records?
 - Are there missing values?
 - Are relationships between tables valid?
 - Are product categories missing?
 - Are customer and seller records valid?
-

Revenue Analysis

- What is the total marketplace revenue?
 - What is the monthly revenue trend?
 - What is the running cumulative revenue?
 - Which months generated the highest revenue?
 - Which months generated the lowest revenue?
-

Product Analysis

- Which categories generate the highest revenue?
 - Which categories generate the lowest revenue?
 - What percentage of revenue does each category contribute?
 - What percentage of categories contribute 80% of revenue?
 - Which category had the highest month-over-month growth?
 - Which categories consistently appeared in the monthly Top 3?
 - What percentage of monthly revenue was generated by each category?
 - Which category was the top performer in each month?
 - What were the Top 3 categories in each month?
-

Seller Analysis

- Who are the top sellers by revenue?
 - Who are the top sellers by orders?
 - What percentage of revenue comes from the Top 10 sellers?
 - What percentage of sellers contribute 80% of revenue?
 - Which seller generates the highest revenue in each state?
 - Who are the Top 3 sellers in each state?
 - How are sellers ranked within their own states?
 - What percentage of revenue does each seller contribute?
-

Customer Analysis

- How many repeat customers exist?
 - How many one-time customers exist?
 - How much revenue comes from repeat customers?
 - What percentage of revenue comes from the Top 10 customers?
 - How should customers be segmented based on spending?
 - Which customers are Champions, Loyal, or At Risk?
 - What is the Recency, Frequency, and Monetary profile of each customer?
-

SQL Concepts Demonstrated

SQL Fundamentals

- SELECT
- WHERE
- GROUP BY
- HAVING
- ORDER BY
- Aggregate Functions

Joins

- INNER JOIN
- LEFT JOIN
- Multi-Table Joins

Conditional Logic

- CASE WHEN

Subqueries

- Scalar Subqueries
- Nested Queries

Common Table Expressions (CTEs)

- Single CTE
- Multi-Step CTE Pipelines

Window Functions

- SUM() OVER()
- LAG()
- RANK()
- DENSE_RANK()
- ROW_NUMBER()
- NTILE()

Advanced Analytics

- Running Totals
 - Ranking
 - Percent Contribution Analysis
 - Pareto Analysis (80/20 Rule)
 - Customer Segmentation
 - Seller Segmentation
 - RFM Analysis
 - Month-over-Month Growth Analysis
 - Time-Series Analysis
-

Key Business Findings

1. Revenue Is Highly Concentrated Among Sellers

Only:

17.54% of sellers contribute 80% of total marketplace revenue.

Insight

A relatively small group of sellers drives most of the business revenue.

2. Seller Revenue Follows Pareto Distribution

A small percentage of sellers generates the majority of marketplace revenue.

Insight

Revenue concentration risk exists on the seller side.

3. Revenue Is Well Distributed Across Customers

Top 10 customers contribute only:

~0.5% of total revenue.

Insight

The business is not dependent on a small number of customers.

4. Customer Base Is Highly Diversified

Revenue comes from a large and broad customer base.

Insight

Marketplace revenue is protected from customer concentration risk.

5. Product Categories Follow Pareto Distribution

A relatively small percentage of categories contributes approximately 80% of total revenue.

Insight

Revenue concentration exists at category level.

6. beleza_saude Is the Most Consistent Category

Appeared:

16 times

among monthly Top 3 revenue-generating categories.

Insight

Strongest long-term category performer.

7. relorios_presentes and cama_mesa_banho Are Consistently Strong

Appeared:

12 times each

among monthly Top 3 categories.

Insight

Reliable revenue-generating categories.

8. Category Performance Changes Over Time

Different categories dominate the marketplace in different months.

Insight

Customer preferences evolve over time.

9. Certain Categories Experience Significant Growth Spikes

Month-over-Month analysis identified categories with exceptionally high growth periods.

Insight

Growth opportunities exist in emerging product categories.

10. Revenue Contribution Changes Month-to-Month

Category contribution percentages vary significantly across months.

Insight

Seasonality and shifting customer demand influence category performance.

11. Seller Performance Varies Significantly

Top-performing sellers generate substantially more revenue than average sellers.

Insight

A large performance gap exists between high and low-performing sellers.

12. Customer Spending Behaviors Are Highly Different

Customer segmentation identified:

- High Value Customers
- Medium Value Customers
- Low Value Customers

Insight

Different customer groups require different engagement strategies.

13. RFM Analysis Successfully Identified Champion Customers

Champion customers demonstrate:

- Recent Purchases
- Frequent Purchases
- High Spending

Insight

These customers should be prioritized for retention.

14. At-Risk Customers Were Identified

RFM analysis identified customers with:

- Low Activity
- Low Spending
- Low Purchase Frequency

Insight

Targeted re-engagement campaigns are required.

15. Marketplace Revenue Became More Diversified Over Time

In early periods, a few categories dominated revenue. Over time, category revenue became more distributed.

Insight

The marketplace matured and diversified its revenue streams.

Business Recommendations

1. Protect High-Performing Sellers

Implement:

- Seller Loyalty Programs
- Dedicated Support
- Exclusive Incentives

for top-performing sellers.

Expected Outcome

Protect revenue-generating seller relationships.

2. Invest Heavily in Proven Categories

Focus sales and marketing efforts on:

- beleza_saude
- relorios_presentes
- cama_mesa_banho

Expected Outcome

Maximize return on marketing investment.

3. Promote Fast-Growing Categories

Identify categories experiencing high month-over-month growth and increase:

- Marketing Spend
- Inventory Availability
- Homepage Visibility

Expected Outcome

Accelerate category growth.

4. Build Customer Loyalty Programs

Target high-value customers using:

- Reward Programs
- Personalized Offers
- Exclusive Benefits

Expected Outcome

Increase customer retention and lifetime value.

5. Use RFM-Based Marketing Strategies

Champion Customers

Provide:

- VIP Programs
- Premium Offers
- Early Access Campaigns

At-Risk Customers

Provide:

- Win-Back Campaigns
 - Personalized Discounts
 - Re-Engagement Promotions
-

6. Reduce Seller Revenue Concentration Risk

Develop mid-tier sellers through:

- Seller Training
- Marketing Support
- Performance Incentives

Expected Outcome

Reduce dependency on a small group of sellers.

7. Improve Retention of High-Value Customers

Focus retention strategies on customers with:

- High Frequency
- High Monetary Value

Expected Outcome

Increase long-term marketplace revenue.

8. Develop Category-Specific Marketing Strategies

Since category performance changes over time, marketing investments should be tailored to category behavior.

Expected Outcome

Improved campaign effectiveness.

Project Outcomes

Through this project, advanced SQL techniques were used to:

- Analyze customer behavior
- Analyze seller performance
- Analyze product category performance
- Understand revenue concentration patterns
- Track month-over-month growth
- Perform customer segmentation
- Implement RFM analysis
- Generate actionable business recommendations

The project demonstrates strong proficiency in:

- Advanced SQL
- Business Analytics
- Data Exploration
- Customer Analytics
- Revenue Analytics
- Time-Series Analysis
- Data Storytelling

2-Minute Interview Story

"I performed an end-to-end analysis of an e-commerce marketplace dataset containing more than 100K orders, 96K customers, and 3K sellers. The project involved data quality assessment, customer analytics, seller analytics, product analytics, revenue analysis, Pareto analysis, customer segmentation, and RFM analysis. I used advanced SQL concepts including CTEs, Window Functions, LAG, DENSE_RANK, NTILE, PARTITION BY, and running totals. Some of the key findings were that only 17.54% of sellers generated 80% of total revenue, while customer revenue was highly diversified. I identified consistently high-performing categories such as beleza_saude, built customer value segments using RFM scoring, analyzed revenue concentration patterns, and generated recommendations around seller retention, customer loyalty strategies, and category-focused growth initiatives."

In []:

In []:

In []:

In []: